

Algoritmy a grafy 1 (BI-AG1), Přednáška č. 5

Binomiální haldy

Dušan Knop, Michal Opler, Tomáš Valla

Fakulta informačních technologií
České vysoké učení technické v Praze
<https://courses.fit.cvut.cz/BI-AG1/>



ZS 2025/2026, 20. října 2025

(Verze dokumentu: 14. 12. 2025 19:21)

HeapBuild – Bankéřova metoda

Věta

Operace **HeapBuild** má časovou složitost $O(n)$.

Důkaz.

Na každý vrchol na začátku položíme p penízků. Co dělá každý prvek?

- Porovná se s menším ze synů (2 instrukce), může se s menším prohodit (1 instrukce) a pak se případně probublává dolů (směrem k listu).
- Ukážeme, že za toto zvládne „zaplatit“ jeden z podstromů.
- Když tedy nakonec přesuneme všechny zbylé penízky do kořene aktuálního stromečku, zbyde nám alespoň $p \cdot (k + 1)$ penízků, kde k je vzdálenost kořenu uvažovaného stromečku od listů.

...

HeapBuild – Bankéřova metoda

Důkaz.

Důkaz proběhne indukcí podle k – vzdálenosti od listu.

(Pro jednoduchost předpokládáme, že poslední hladina je zcela zaplněná – tím formálně ukazujeme $2pn$, ale není těžké důkaz poupravit.)

- ZI: $k = 0$. Triviálně platí, neboť neprovedeme žádné porovnání a na kontě zůstane $p = p(k + 1)$.
- IK: $k \leftarrow k - 1$. Máme „nový kořen“ ve stromě T a předpokládáme, že podstromy T_ℓ a T_r jsou korektně vytvořeny a že (IP) každý z jejich kořenů má na kontě pk . Hloubka stromu T je $k + 1$. Všimli jsme si, že v nejhorším případě uděláme s novým prvkem na každé hladině (krom poslední) 3 operace. Celkově tedy za „opravení“ stromu T zaplatíme $3k$. Na konci nám tedy zbyde $z = p + 2pk - 3k$. Pokud zvolíme $p \geq 3$, bude jistě $z \geq p(k + 1)$. □

Binární sčítačka – připomenutí

- **Binární sčítačka** je obvod, který uchovává bitovou reprezentaci čísla uloženou v buňkách nastavitelných na 0 nebo 1.
- Uvažujeme k -bitovou **sériovou sčítačku (ripple-carry)**:
přenos se postupně šíří zprava doleva
- Po zapnutí má sčítačka všechny bity nastaveny na 0.
- Sčítačka podporuje dvě operace:
 - ▶ **Inc**(n) zvýší číslo n reprezentované v sčítačce o 1.
 - ▶ **Add**(n, m) přičte k číslu v sčítačce n binární číslo m .

n :

0	1	0	1
---	---	---	---

n :

1	0	0	1
---	---	---	---

m :

0	0	1	1
---	---	---	---

Inc(n):

0	1	1	0
---	---	---	---

Add(n, m):

1	1	0	0
---	---	---	---

Binomiální minimová halda

- Pro přehlednost zkracujeme jako BH.
- Podporuje stejné operace jako binární halda.
- Navíc je schopna rychle provádět operaci sloučení dvou hald, která má u binární haldy lineární složitost (sloučit dvě binární haldy velikostí m a n má složitost operace **HeapBuild** haldy velikosti $m + n$).
- Binomiální halda patří do rodiny tzv. **mergeable heaps**.
- Další dobrou vlastností je vynikající amortizovaná složitost operace vkládání.
- Nevýhodou jsou násobně vyšší paměťové nároky než u binární haldy.

Binomiální minimová halda

BHInsert	$O(\log n), \Theta^*(1)$	Vloží do BH nový prvek.
BHFindMin	$O(1)$	Vrátí minimum množiny prvků BH.
BHExtractMin	$O(\log n)$	Odstraní z BH minimum množiny jejích prvků.
BHMerge	$O(\log n)$	Sloučí dvě BH do jedné.
BHBuild	$O(n)$	Postaví z n prvků BH.
BHDecreaseKey	$O(\log n)$	Sníží hodnotu klíče prvku BH.
BHIncreaseKey	$O(\log n)$	Zvýší hodnotu klíče prvku BH.
BHDelete	$O(\log n)$	Smaže prvek BH.

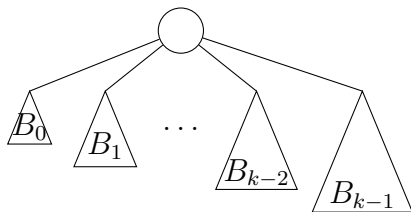
Poznámka: Implicitně budeme nadále předpokládat minimové BH.

Binomiální strom

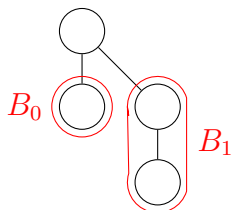
Definice

Binomiální strom řádu k (značíme B_k) je uspořádaný (t.j. záleží na pořadí synů) zakořeněný strom, pro který platí:

- 1 B_0 je tvořen pouze kořenem.
- 2 Pro $k \geq 1$ získáme B_k ze stromů $B_0, B_1, \dots, B_{k-1}$ tak, že přidáme nový kořen a kořeny těchto stromů uděláme (takto popořadě) syny nového kořene.



(a) B_k



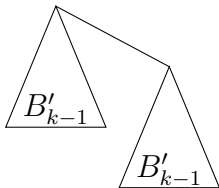
(b) B_2

Binomiální strom – Alternativní definice

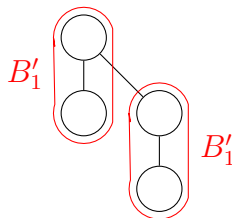
Definice

Binomiální strom řádu k (značíme B'_k) je uspořádaný zakořeněný strom, pro který platí:

- B'_0 je tvořen pouze kořenem.
- Pro $k \geq 1$ se B'_k skládá ze stromu B'_{k-1} , pod jehož kořenem je jako nejpravější syn napojený další strom B'_{k-1} .



(a) B'_k

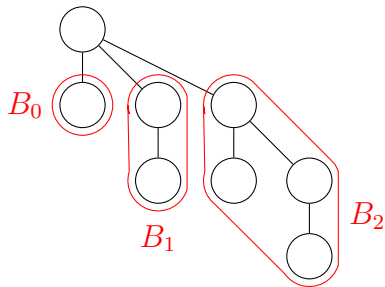


(b) B'_2

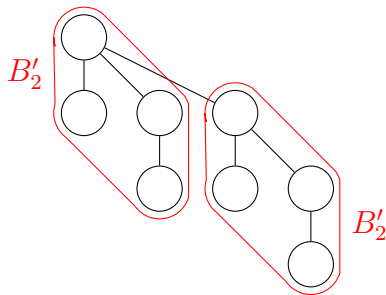
Ekvivalence definic

Lemma

Stromy B_k a B'_k jsou izomorfní.



(a) B_3



(b) B'_3

Ekvivalence definic

Lemma

Stromy B_k a B'_k jsou izomorfní.

Důkaz $B_k \Rightarrow B'_k$.

- Matematickou indukcí podle k .
- Pro $k = 0$ tvrzení zjevně platí.
- Pod kořenem stromu B_k jsou dle jeho definice zavěšeny stromy $B_0, \dots, B_{k-1}$.
- Odtržením nejpravějšího podstromu B_{k-1} od B_k však dostáváme podle indukčního předpokladu strom B_{k-1} .
- To dává přesně definici stromu B'_k . □

Ekvivalence definic - dokončení důkazu

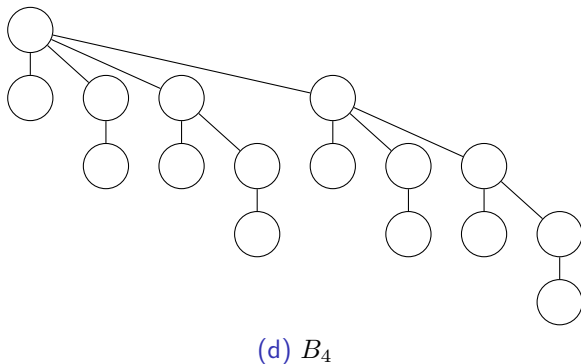
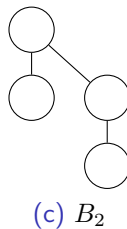
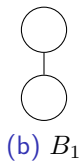
Lemma

Stromy B_k a B'_k jsou izomorfní.

Důkaz $B'_k \Rightarrow B_k$.

- Naopak, uvážíme-li strom B'_k , z indukce vyplývá, že B'_{k-1} je izomorfní s B_{k-1} , pod jehož kořen jsou dle definice napojeny stromy $B_0, \dots, B_{k-2}$.
- Pod kořen B'_k jsou tudíž napojeny stromy $B_0, \dots, B_{k-1}$.
- To dává přesně definici stromu B_k . □

Příklady binomiálních stromů



Vlastnosti binomiálních stromů

Lemma

Počet hladin stromu B_k je roven $k + 1$, stupeň kořene je k , a počet vrcholů B_k je roven 2^k .

Důkaz.

- Indukcí podle k .
- Strom B_0 má jistě 1 hladinu a $2^0 = 1$ vrchol.
- Z indukčního předpokladu vyplývá, že počet hladin B_{k-1} je k a počet vrcholů je 2^{k-1} .
- Užitím předchozího lemmatu dostáváme, že strom B_k je složený ze dvou stromů B_{k-1} , z nichž jeden je o hladinu níže než druhý, což dává počet hladin $k + 1$ stromu B_k .
- Složením dvou stromů B_{k-1} dostáváme $2 \cdot 2^{k-1} = 2^k$ vrcholů.
- Stupeň kořene B_{k-1} je dle IP $k - 1$ a přidáním jednoho nového syna je stupeň B_k tedy roven k . □

Vlastnosti binomiálních stromů

Důsledek

Binomiální strom s n vrcholy (pokud existuje) má $1 + \log n$ hladin a počet synů kořene $\log n$.

- Proč se těmito speciálním stromům říká **binomiální**?

Lemma

Počet vrcholů stromu B_k na i . hladině ($i \in \{0, \dots, k\}$) je $n_k(i) = \binom{k}{i}$.

Lemma

Počet vrcholů stromu B_k na i . hladině ($i \in \{0, \dots, k\}$) je $n_k(i) = \binom{k}{i}$.

Důkaz.

- Indukcí podle řádu k .
- Lemma platí triviálně pro B_0 a B_1 (a B_2) (viz Slajd 12).
- Necht' tedy $k \geq 2$.
- Z definice B'_k plyne, že vrcholy B'_k na i . hladině, $0 < i < k$, jsou tvořeny:
 - ▶ vrcholy levého B'_{k-1} na i . hladině a
 - ▶ vrcholy pravého B'_{k-1} na $(i-1)$. hladině.
- Z indukčního předpokladu tedy dostaneme Pascalovým pravidlem

$$n_k(i) = n_{k-1}(i) + n_{k-1}(i-1) = \binom{k-1}{i} + \binom{k-1}{i-1} = \binom{k}{i}.$$



Definice binomiální haldy

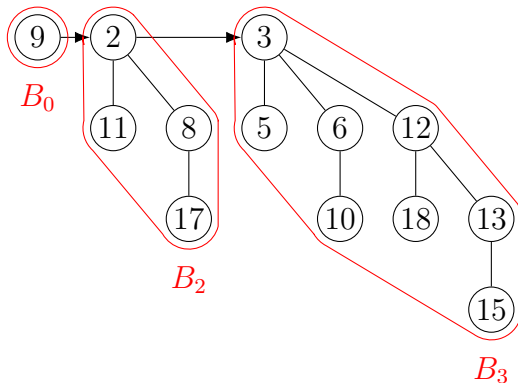
Definice

Binomiální halda (BH) obsahující n prvků je **uspořádaná** množina binomiálních stromů $\mathcal{T} = T_1, \dots, T_\ell$, kde platí:

- Stromy T_i jsou v $\mathcal{T}$ uspořádány **vzestupně** podle svých **řádů**.
- $n = |V(T_1)| + \dots + |V(T_\ell)|$.
- Pro každé nezáporné k se v množině $\mathcal{T}$ vyskytuje **nejvýše jeden** binomiální strom řádu k .
- Každý vrchol v v každém stromu obsahuje klíč $k(v)$.
- Pro každý strom T_i platí **haldové uspořádání** klíčů, čili $\forall v \in V(T_i)$ a pro všechny jeho syny $s_j, j = 1, 2, \dots, m$, platí $k(v) \leq k(s_j)$.

Příklad BH

BH velikosti $n = 13$ prvků:



K zamyšlení

- Rozmyslete, proč je struktura množiny **řadů** binomiálních stromů BH určena jednoznačně její velikostí n ?

Paměťová reprezentace BH v počítači

- Pro uložení uspořádané množiny stromů $\mathcal{T}$ BH se používá **spojový seznam**.
- Seznamy synů jednotlivých vrcholů v binomiálních stromech budeme také udržovat ve spojových seznamech.
- Konkrétní implementaci prvku BH si předvedeme později, až budeme znát požadavky na operace, které s BH budeme provádět.
- Klíč prvku v budeme v pseudokódu značit $k(v)$.

Vlastnosti BH

Tvrzení

Binomiální strom B_i se vyskytuje v seznamu $\mathcal{T}$ n -prvkové BH právě tehdy, když ve dvojkovém zápisu $b_k b_{k-1} \dots b_0$ čísla n je $b_i = 1$.

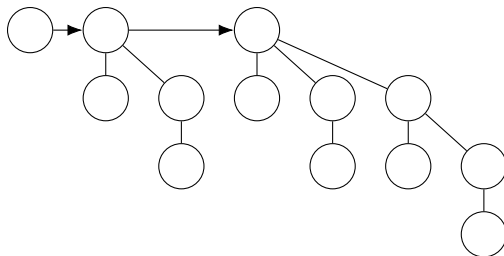
Důkaz.

Protože v BH nelze použít dva binomiální stromy stejného řádu a každý binomiální strom B_i přispěje do n -prvkové BH právě svými $|V(B_i)| = 2^i$ vrcholy, je poskládání n prvkové BH z binomiálních stromů ekvivalentní zápisu čísla n ve standardní dvojkové soustavě. □

Vlastnosti BH

Důsledek

n -prvková BH má až $O(\log n)$ binomiálních stromů.



Struktura BH pro $n = 13$; $(13)_2 = 1101$

Nalezení minima množiny prvků BH – BHFindMin

Operace **BHFindMin**(H):

- Minimum celé BH se musí nacházet v jednom z kořenů stromů T_i .
- Stačí projít seznam $\mathcal{T}$, což bude trvat čas $O(\log n)$.
- Používáme-li tuto funkci často, vyplatí se udržovat ukazatel na tento globálně nejmenší kořen. Operaci **BHFindMin** lze pak provést v konstantním čase.

Sloučení dvou BH – BHMerge

- **Sloučení** dvou BH popíšeme nejdříve, protože se pomocí něho realizují ostatní operace.
- Operace **BHMerge** ze dvou BH vytvoří jedinou, obsahující sjednocení prvků obou vstupních BH.
- Nejprve popíšeme operaci **BHMergeTree**, která slije dohromady dva binomiální stromy stejného řádu B_i a vytvoří strom B_{i+1} (na to lze nahlížet i jako na sloučení dvou BH tvořených jediným binomiálním stromem stejného řádu).

Slití dvou binomiálních stromů

Algoritmus **BHMergeTree**

Vstup: Binomiální stromy T_1 , T_2 takové,
že $\text{řád}(T_1) = \text{řád}(T_2)$

Výstup: Výsledný strom T_{out}

- (1) Pokud $k(\text{kořen}(T_1)) \leq k(\text{kořen}(T_2))$:
- (2) Připoj $\text{kořen}(T_2)$ jako nejpravějšího syna
 pod $\text{kořen}(T_1)$.
- (3) $T_{out} := T_1$
- (4) Jinak:
- (5) Připoj $\text{kořen}(T_1)$ jako nejpravějšího syna
 pod $\text{kořen}(T_2)$.
- (6) $T_{out} := T_2$

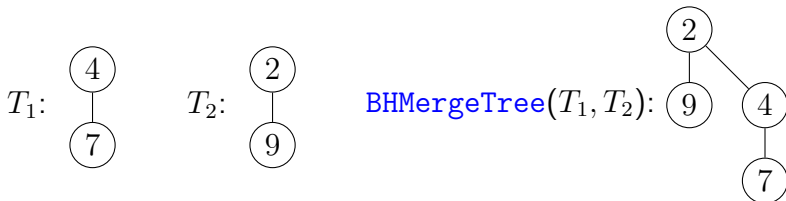
Slití dvou binomiálních stromů stejného řádu

Pozorování

Algoritmus **BHMergeTree**(T_1, T_2), kde $\text{řád}(T_1) = \text{řád}(T_2)$, vytvoří korektní binomiální strom s řádem $\text{řád}(T_1) + 1$.

Tvrzení

BH lze implementovat tak, že **BHMergeTree**(T_1, T_2) má časovou složitost $O(1)$.



Než ukážeme jak reprezentovat prvky BH, „posbíráme“ všechny požadavky na rychlé operace.

Idea algoritmu BHMerge pro sloučení dvou BH

- Mějme BH A a B , kde počet prvků A je a a v binárním zápise $a = a_k a_{k-1} \dots a_0$ a počet prvků B je $b = b_\ell b_{\ell-1} \dots b_0$.
- Výsledná halda C bude mít $c = a + b$ prvků, $c = c_m c_{m-1} \dots c_0$.
- Řády binomiálních stromů ve všech tří haldách jsou jednoznačně určeny binárním zápisem jejich počtu prvků.
- Připomeňme „školní“ algoritmus na sčítání binárních čísel a a b pod sebou:
jdeme od nejnižších řádů binárního zápisu k nejvyšším a
 - ▶ výsledný bit $c_i = (a_i + b_i + carry_i) \bmod 2$, kde $carry_i$ je tzv. přenos z předchozího řádu a
 - ▶ následně $carry_{i+1} := (a_i + b_i + carry_i) \div 2$.

Binární sčítání - příklad

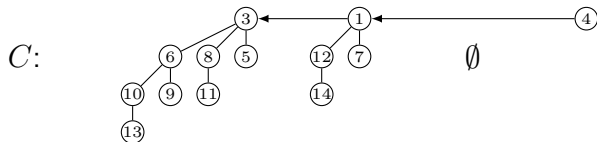
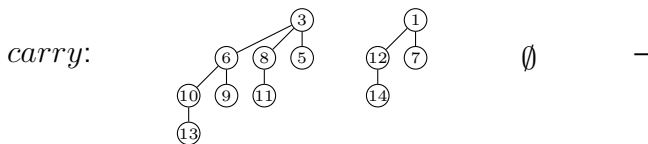
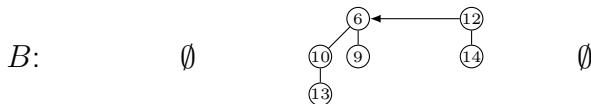
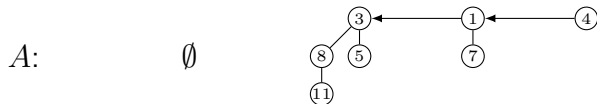
- $c_i = (a_i + b_i + carry_i) \bmod 2$
- $carry_{i+1} := (a_i + b_i + carry_i) \div 2$

a	=				1	0	0	1	0
b	=	1	0	1	1	1	1	1	0
$carry$		0	1	1	1	1	1	0	
c	=	1	1	0	1	0	0	0	0

Aplikace binárního sčítání na BH

- Algoritmus na vytvoření BH C nyní zrcadlí předchozí algoritmus sčítání binárních čísel s tím rozdílem, že:
 - 1 Bity $a_i, b_i, carry_i$ budou nyní binomiální stromy $A_i, B_i, carry_i$ řádu i nebo prázdné.
 - 2 Místo sčítání dvou jedničkových bitů voláme **BHMergeTree** na dva binomiální stromy stejného řádu, což vytvoří strom vyššího řádu a tedy přenos $carry$ do dalšího řádu.
 - 3 Jsou-li všechny tři stromy $A_i, B_i, carry_i$ neprázdné, sloučíme A_i a B_i a výsledek se stane přenosem do vyššího řádu $carry_{i+1}$ a do C_i přiřadíme $carry_i$.
 - 4 Protože udržujeme seznamy binomiálních stromů BH uspořádané dle jejich řádů, lze algoritmus realizovat průchodem dvou ukazatelů po těchto seznamech a přeskokováním řádů stromů nepřítomných v BH.

Aplikace binárního sčítání na BH



Pro přesnost ještě ukážeme zmíněný algoritmus v pseudokódu.

Algoritmus **BHMerge**(A, B)

```
(1) scitanci[0...2]                //pole aktuálních sčítanců
(2) carry := null                  //přenos
(3) neprazdnych := 2              //počet zbývajících sčítanců
(4) akt_rad := 0                   //aktuální řád
(5) Dokud neprazdnych  $\geq 2$ :
(6)   neprazdnych := 0
(7)   pocet := 0                   //počet sčítanců v aktuálním řádu
(8)   Pokud  $A$  je neprázdná:
(9)     neprazdnych++
(10)     $a :=$  strom nejnižšího řádu v  $A$ 
(11)    Pokud  $\text{řád}(a) = \text{akt\_rad}$ :
(12)      scitanci[pocet] :=  $a$ 
(13)      pocet++
(14)      vyřaď  $a$  z  $A$ 
...

```

```
(15)   Pokud  $B$  je neprázdná:
(16)       neprazdnych++
(17)        $b := \text{strom nejnižšího řádu v } B$ 
(18)       Pokud  $\text{řád}(b) = \text{akt\_rad}$ :
(19)           scitanci[pocet] :=  $b$ 
(20)           pocet++
(21)           vyřaď  $b$  z  $B$ 
(22)   Pokud carry není null:
(23)       neprazdnych++
(24)       scitanci[pocet] := carry
(25)       pocet++
(26)       carry := null
```

...

```
(27)   Pokud pocet= 3:  
(28)       zapiš scitanci[2] do výstupu C  
(29)       carry := BHMergeTree(scitanci[0], scitanci[1])  
(30)   Pokud pocet= 2:  
(31)       carry := BHMergeTree(scitanci[0], scitanci[1])  
(32)   Pokud pocet= 1:  
(33)       zapiš scitanci[0] do výstupu C  
(34)   akt_rad++  
(35)   Přeřaď do C zbytek BH A nebo B  
       (pokud ještě něco zbývá)  
(36)   Přepoj ukazatel na minimum na menší z minim z A a B
```

Analýza operace BHMerge

Věta

Algoritmus **BHMerge** je korektní a jeho časová složitost je $O(\log n)$.

Důkaz.

- V každé iteraci se zpracují všechny stromy řádu akt_rad .
- Následně se akt_rad zvýší o jedna.
- Všechny operace uvnitř cyklu trvají čas $O(1)$.
- Nejvyšší řád je $O(\log n)$, celkový čas je tedy $O(\log n)$. □

Vložení nového prvku do BH – BHInsert

Operace **BHInsert**(H, x):

- (1) Vytvoříme BH H' s jediným prvkem x .
- (2) **BHMerge**(H, H')



Analýza operace BHInsert

Tvrzení

Operace **BHInsert** má v nejhorším případě složitost $O(\log n)$. Pro na počátku prázdnou BH trvá posloupnost n volání operace **BHInsert** čas $O(n)$. **BHInsert** má tedy amortizovanou časovou složitost $O^*(1)$.

Důkaz.

- Víme, že binomiální stromy tvořící n -prvkovou BH přesně odpovídají jedničkovým bitům v dvojkovém zápisu čísla n .
- Operace **BHInsert** je sloučení s jednoprvkovou BH a to odpovídá operaci **Inc** binární sčítačky.
- Provedení operace **BHMergeTree** s $O(1)$ složitostí během **BHInsert** odpovídá bitové inverzi v operaci **Inc**.
- Dle analýzy binární sčítačky má tedy operace **BHInsert** amortizovanou složitost $O^*(1)$. □

Vytvoření n prvkové BH – BHBuild

- Voláním **BHInsert** n krát po sobě vytvoříme BH o velikosti n .
- Podle předchozí analýzy bude trvat vytvoření n -prvkové BH čas $O(n)$.

Odstranění minima z BH – BHExtractMin

Algoritmus **BHExtractMin**(H)

- (1) Najdi v BH H strom T , jehož kořen je minimem.
- (2) Odpoj tento strom T z BH H .
- (3) Odtrhni z T jeho kořen.
- (4) Z T vytvoř novou BH H' .
- (5) Proveď **BHMerge**(H, H').

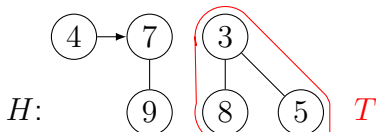
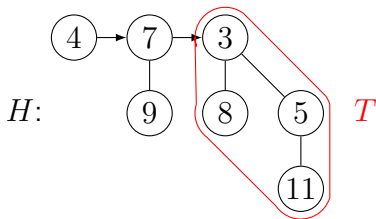
Z T vytvoř novou BH H'

- Potřebujeme v čase $O(\log n)$ vytvořit BH ze synů (podstromů) stromu T .
- Toho potřebujeme dosáhnout při zachování $O(1)$ času pro **BHMergeTree**.

Operace BHExtractMin

Algoritmus $\text{BHExtractMin}(H)$

- (1) Najdi v BH H strom T , jehož kořen je minimem.
- (2) Odpoj tento strom T z BH H .
- (3) Odtrhni z T jeho kořen.
- (4) Z T vytvoř novou BH H' .
- (5) Proveď $\text{BHMerge}(H, H')$.



Analýza operace BHEextractMin

Tvrzení

Časová složitost operace **BHEextractMin** je $O(\log n)$.

Důkaz.

- Kroky (1) a (2) trvají čas $O(1)$, stačí si udržovat minimový ukazatel na strom a vypojit ho ze seznamu.
- Kroky (3) a (4) trvají $O(\log n)$, protože kořen binomiálního stromu má nejvýše $O(\log n)$ synů.
- Sloučení hald trvá $O(\log n)$, včetně rekonstrukce minimového ukazatele. □

Paměťová reprezentace prvků BH

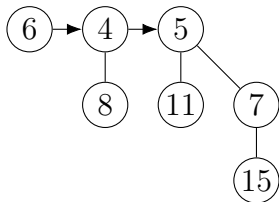
- Prvek v BH bude v počítači reprezentován pomocí následující struktury:

ukazatel na otce
hodnota $k(v)$
ukazatel na levého sourozence
ukazatel na nejpravějšího syna

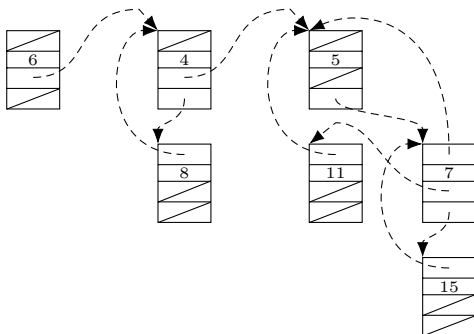
Tvrzení

BHMergeTree i vytvoření BH ze seznamu synů kořene lze v čase $O(\log n)$, kde n je počet prvků ve výsledné BH.

Paměťová reprezentace BH



(a) Binomiální halda H



(b) Reprezentace binom. haldy H

K zamyšlení

Jak v minimové BH udělat operace:

- **BHDecreaseKey** v čase $O(\log n)$?
 - **BHDelete** v čase $O(\log n)$?
 - **BHIncreaseKey** v čase $O(\log n)$?
-
- (Všechny operace dostanou ukazatel na prvek, se kterým se pracuje.)
 - Bude na cvičení.

Srovnání binární a binomiální haldy

- Operace vložení prvku do haldy velikosti n :
 - ▶ Binární: $O^*(\log n)$, $O(\log n)$.
 - ▶ Binomiální: $O^*(1)$, $O(\log n)$.
- Operace **ExtractMin** z haldy velikosti n :
 - ▶ $O(\log n)$ v obou případech.
- Operace sloučení 2 hald velikosti n :
 - ▶ Binární: $O(n)$ (viz operace **HeapBuild** v minulé přednášce).
 - ▶ Binomiální: $O(\log n)$.
- Binomiální haldy jsou nejjednodušším řešením tzv. **mergeable heaps**, které dokážou velmi efektivně provést operaci sloučení a ostatní operace mají na operaci sloučení postavené.

Co bychom měli po dnešku znát

- Binomiální stromy a jejich vlastnosti.
- Binomiální haldy a operace s nimi.